Proyecto 2: Juego de reflejos en assembly sobre Pochoco SoC

ICC3302 Arquitectura de Computadores, 2026-2

Integrantes: _____

Octubre 2026

1. Introducción

En este proyecto implementamos un juego de reflejos que corre sobre el Espino Core, el procesador del Pochoco SoC, sintetizado en la FPGA Lattice iCE40 HX1K de la Nandland Go Board.

El trabajo tiene tres partes que se conectan entre sí:

1. **Hardware.** Modificamos el SoC para agregar un contador de ciclos de reloj que el programa pueda leer, y creamos un módulo superior propio (`game_top.v`) que es el que se sintetiza.
2. **Assembler.** Escribimos en Python un programa (`assembler/asm.py`) que traduce el código assembly a código máquina, es decir, a las palabras de 32 bits que entiende el procesador. No usamos el assembler oficial de RISC-V.
3. **Juego.** Escribimos el juego en assembly (`sw/game.s`). El assembler lo convierte en `sw/game.hex`, y ese archivo se carga en la memoria del SoC cuando se sintetiza el diseño.

El recorrido completo es el siguiente: `game.s` pasa por nuestro assembler y se convierte en `game.hex`; luego yosys, nextpnr e icepack generan el bitstream con el procesador y el programa ya cargado en la memoria; finalmente iceprog lo graba en la placa.

2. Arquitectura del sistema

2.1 Qué es cada pieza

Espino Core es el procesador. Implementa RV32E, que es la versión de RISC-V de 32 bits con 16 registros (x0 a x15). Tiene una particularidad importante: las instrucciones de desplazamiento (shifts) se decodifican pero están deshabilitadas en la ALU para ahorrar espacio en la FPGA, así que en la práctica se ejecutan como una suma. Además RV32E no incluye multiplicación ni división. Estas dos limitaciones influyeron en cómo escribimos el juego.

Pochoco SoC conecta el procesador con una memoria RAM (donde están juntos el programa y los datos) y con los periféricos de la placa: displays de 7 segmentos, LEDs y botones.

game_top es el módulo que pide el enunciado. No tiene lógica propia: solo instancia el `pochoco_soc`, le indica qué archivo de memoria cargar (`sw/game.hex`) y conecta sus puertos a los pines de la placa. Como usamos los mismos nombres de puertos que el archivo `goboard.pcf`, no fue necesario cambiar la asignación de pines. En el `Makefile` cambiamos `TOP := game_top` para que se sintetice este módulo.

2.2 Cómo se comunica el programa con la placa

El procesador no tiene instrucciones especiales para prender un LED o leer un botón. En cambio, se usa I/O mapeada en memoria: ciertas direcciones no corresponden a la RAM sino a registros de los periféricos. El SoC mira el bit 31 de la dirección; si está en 1, la lectura o escritura va a los periféricos en vez de a la RAM.

Dirección	Registro	Acceso
0x8000_0000	Displays (un byte, cada nibble es un dígito)	escritura
0x8000_0004	LEDs (4 bits)	escritura
0x8000_0008	Botones (4 bits, 1 = apretado)	lectura
0x8000_000C	Contador de ciclos (agregado por nosotros)	lectura

Por ejemplo, para prender todos los LEDs el programa guarda la dirección base en un registro y escribe el valor 15 (1111 en binario) con un store:

```
lui  x2, 0x80000      # x2 = 0x80000000
addi x3, x0, 15       # x3 = 1111
sw   x3, 4(x2)        # escribe en 0x80000004 -> se prenden los 4 LEDs
```

2.3 Modificación al RTL: contador de ciclos

Para medir tiempos el programa necesita saber cuánto tiempo ha pasado. Agregamos en `pochoco_periph.v` un registro de 32 bits que aumenta en 1 en cada flanco de subida del reloj y se reinicia junto con el resto del sistema:

```
reg [31:0] cyc_q;
always @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) cyc_q <= 32'b0;
  else         cyc_q <= cyc_q + 32'd1;
end
```

y lo agregamos al bloque de lectura del periférico:

```
6'd3: rdata_o <= cyc_q;   // contador de ciclos (offset 0x0C)
```

El periférico elige el registro usando `addr[7:2]`, que es la dirección dividida por 4. Por eso el offset 0x0C corresponde al caso 3 (los botones, en 0x08, son el caso 2). Desde el programa el contador se lee con `lw x?, 12(x2)`.

Cómo se mide un intervalo. Se lee el contador al principio, se vuelve a leer al final y se restan los dos valores. El resultado es la cantidad de ciclos que pasaron.

Qué pasa cuando el contador se desborda. Un registro de 32 bits llega a su máximo y vuelve a 0 cada 2^32 / 25.000.000 = 171,8 segundos. Esto no es un problema: la resta se hace en módulo 2^32, así que aunque el contador haya dado la vuelta entre las dos lecturas, la diferencia sigue siendo correcta. Lo único que se necesita es que el intervalo medido sea menor a 171 segundos, y todos los del juego lo son.

Por qué en hardware. Otra opción era contar vueltas de un loop, pero el tiempo que tarda cada instrucción en el Espino no es constante (los accesos a memoria tienen esperas), así que la medición sería imprecisa. Con el contador el tiempo es exacto al ciclo.

2.4 Recursos de la FPGA

Después de la síntesis y el place & route, el diseño completo usa:

Recurso	Uso
Celdas lógicas (LUT de 4 entradas + flip-flop)	1159 de 1280 (90 %)
Bloques de memoria (SB_RAM40_4K)	12 de 16 (75 %)
Frecuencia máxima estimada por nextpnr	43 MHz (la placa funciona a 25 MHz)

Como quedaban pocas celdas libres, decidimos no agregar más hardware (por ejemplo un divisor o habilitar los shifts) y resolver las divisiones por software.

3. Temporización

3.1 Frecuencia de reloj y constantes

La Go Board tiene un oscilador de **25 MHz**. Eso significa 25.000.000 de ciclos por segundo, y cada ciclo dura 1 / 25.000.000 = 40 ns. A partir de esto calculamos todas las constantes del juego, que están definidas con `.equ` al principio de `game.s` para poder cambiarlas fácilmente:

Constante	Ciclos	Cálculo	Para qué se usa
CICLOS_3S	75.000.000	3 s × 25.000.000	fase con los 4 LEDs prendidos
CICLOS_DECIMA	2.500.000	0,1 s × 25.000.000	unidad en que se muestra el tiempo
CICLOS_ERROR	25.000.000	1 s × 25.000.000	tiempo mostrando “EE”
CICLOS_REBOTE	500.000	0,02 s × 25.000.000	antirrebote de los botones
CICLOS_PARPADEO	6.250.000	0,25 s × 25.000.000	parpadeo de los LEDs al final

3.2 Espera de tres segundos

Todas las esperas se hacen con la subrutina `esperar`, que recibe en x6 la cantidad de ciclos:

```
esperar:
    lw   x14, 12(x2)          # t0 = contador al empezar
esperar_loop:
    lw   x15, 12(x2)          # t = contador ahora
    sub  x15, x15, x14        # t - t0 = ciclos que han pasado
    bltu x15, x6, esperar_loop  # si todavía no llega a x6, sigue esperando
    ret
```

Se usa `bltu` (comparación sin signo) porque una cantidad de ciclos nunca es negativa. Para la fase inicial se carga `x6 = CICLOS_3S` y se llama a `esperar`. La espera dura 75.000.000 ciclos más lo que tarde la última vuelta del loop (unos pocos ciclos). Después se llama a `soltar`, que espera que ningún botón esté apretado. Por lo tanto la fase con los cuatro LEDs dura **al menos** 3 segundos, como pide el enunciado.

3.3 Conversión de ciclos a décimas de segundo

Una décima de segundo son 2.500.000 ciclos, entonces:

```
décimas = parte entera de (ciclos / 2.500.000)
```

Como el Espino no tiene instrucción de división, la hacemos con restas sucesivas: mientras queden al menos 2.500.000 ciclos, se restan y se suma 1 al resultado.

```
    sub  x5, x4, x12          # ciclos = t_final - t_inicio
    addi x13, x0, 0           # décimas = 0
    li   x7, CICLOS_DECIMA
a_decimas:
    bltu x5, x7, decimas_listo  # si quedan menos de 2.500.000, terminamos
    sub  x5, x5, x7
    addi x13, x13, 1
    j    a_decimas
```

Por ejemplo, si el jugador se demora 9.300.000 ciclos (0,372 s), el loop resta 3 veces y quedan 1.800.000 ciclos, que son menos de una décima. El resultado es 3, y en el display aparece “03”. O sea, el resultado se trunca. El loop da tantas vueltas como décimas tenga el resultado (menos de 100 para tiempos normales), y como se ejecuta después de tomar el tiempo final, no afecta la medición.

4. Programa en assembly

4.1 Uso de los registros

Registro	Uso
x1	dirección de retorno de las subrutinas (call / ret)
x2	dirección base de los periféricos, 0x8000_0000
x3, x4, x5, x7	temporales
x6	argumento de `esperar` (cantidad de ciclos)
x8	cantidad de rondas correctas
x9	suma de los tiempos en décimas (para el promedio)
x10	estado del generador pseudoaleatorio
x11	máscara del LED objetivo (1, 2, 4 u 8)
x12	valor del contador al empezar la medición
x13	tiempo de la ronda en décimas
x14, x15	temporales de las subrutinas

Las subrutinas (`esperar`, `soltar` y `mostrar`) no llaman a otras subrutinas. Gracias a eso x1 nunca se sobrescribe antes de un `ret` y no necesitamos usar el stack.

4.2 Funcionamiento paso a paso

Inicio. Al encender la placa, el programa deja el display en “00” y los LEDs apagados, y espera que se apriete cualquier botón. Ese momento se usa como semilla para elegir los LEDs al azar (se explica en la sección 4.3). Luego pone en 0 las rondas correctas (x8) y la suma de tiempos (x9).

Fase inicial de la ronda. Se prenden los cuatro LEDs escribiendo 15 en el registro de LEDs, se esperan 3 segundos con `esperar` y luego se llama a `soltar`, para que nadie pueda tener un botón apretado de antes cuando aparezca el LED objetivo.

Selección y encendido del LED objetivo. Se elige un LED al azar, su máscara queda en x11 y se escribe en el registro de LEDs. En la instrucción inmediatamente siguiente se lee el contador y se guarda en x12: ahí empieza la medición.

```
    sw   x11, 4(x2)          # prende solo el LED objetivo
    lw   x12, 12(x2)         # aquí empieza la medición
```

Así la medición parte en el mismo instante lógico en que comienza la ronda para el jugador. La diferencia entre ambas instrucciones es de menos de 1 µs, que es despreciable frente a una décima.

Espera de la pulsación. El programa lee los botones en un loop hasta que alguno esté apretado. Apenas eso ocurre, lee el contador (tiempo final) y compara los botones con la máscara del LED:

```
espera_boton:
    lw   x3, 8(x2)           # lee los botones
    beq  x3, x0, espera_boton # si no hay ninguno apretado, sigue esperando
    lw   x4, 12(x2)          # tiempo al apretar
    bne  x3, x11, error      # si no coincide con el LED, es un error
```

Pulsación correcta. Se calcula el tiempo en décimas (sección 3.3), se suma a x9, se suma 1 a x8 y se muestra el tiempo en el display. Si todavía no hay 10 rondas correctas, se vuelve a la fase inicial de una nueva ronda.

Pulsación incorrecta. Si el valor de los botones no es igual a la máscara (esto incluye apretar otro botón o varios a la vez), el programa salta a `error`: muestra “EE” en el display, apaga los LEDs, espera que suelten el botón, espera 1 segundo y vuelve a la fase inicial de la misma ronda. Como x8 y x9 no se modifican en este camino, las rondas correctas acumuladas se conservan.

Fin de la partida. Después de la décima ronda correcta se calcula el promedio (sección 4.5) y se muestra en el display. Los LEDs quedan parpadeando cada 0,25 s para indicar que terminó la partida. Si se aprieta un botón, empieza una partida nueva.

Antirrebote. Los botones mecánicos rebotan: al soltarlos la señal puede pasar varias veces entre 0 y 1 durante algunos milisegundos. Si no se considera, un rebote podría contarse como una segunda pulsación. La subrutina `soltar` espera que los botones estén en 0 durante 20 ms seguidos; si durante ese tiempo aparece un 1, vuelve a empezar. Se llama después de cada pulsación.

4.3 Generación pseudoaleatoria

En cada ronda se lee el contador de ciclos, se suma al estado x10 y del resultado se toman los bits 9 y 10:

```
    lw   x3, 12(x2)
    add  x10, x10, x3
    andi x3, x10, 0x600      # deja solo los bits 9 y 10
```

El resultado puede ser 0x000, 0x200, 0x400 o 0x600, y según cuál sea se elige el LED 1, 2, 3 o 4. Como no hay shifts para mover esos bits a la posición 0, la elección se hace comparando con `bltu` contra 0x200, 0x400 y 0x600.

De dónde viene el azar. La semilla inicial es el valor del contador cuando el jugador aprieta el botón para empezar. En cada ronda se le suma de nuevo el contador, y ese valor depende de cuánto se demoró el jugador en la ronda anterior. Una persona no puede repetir sus tiempos con precisión de microsegundos, así que cada partida produce una secuencia distinta.

Por qué los bits 9 y 10 y no los más bajos. Los loops de espera del programa tienen un período fijo de algunos ciclos. Por eso el contador siempre se lee en posiciones parecidas respecto a ese período, y los bits 0 y 1 saldrían casi siempre iguales. Los bits 9 y 10 cambian cada 512 ciclos (unos 20 µs), mientras que la variación del tiempo de una persona es de milisegundos. En ese rango los cuatro valores son prácticamente igual de probables.

Por qué se espera un botón al inicio. Si el juego empezara solo después del reset, todo lo que pasa hasta la primera ronda sería siempre igual y el primer LED sería siempre el mismo. Esperar al jugador hace que incluso la primera ronda sea aleatoria.

4.4 Mostrar el tiempo en el display

El periférico de display toma un byte y muestra cada mitad (nibble) como un dígito hexadecimal. Si se escribe el número 37 directamente, que en hexadecimal es 0x25, el display mostraría “25”. Para que muestre “37” hay que escribir 0x37, o sea, el número en BCD: decenas × 16 + unidades.

La subrutina `mostrar` hace esta conversión:

1. Si el valor es mayor que 99, lo deja en 99 (el display solo tiene dos dígitos, así que el máximo es 9,9 s).
2. Calcula las decenas restando 10 mientras se pueda; lo que queda son las unidades.
3. Multiplica las decenas por 16. Como no hay shifts ni multiplicación, suma el registro consigo mismo cuatro veces (×2, ×4, ×8, ×16).
4. Suma las unidades y escribe el resultado en el display.

Ejemplo: para 37, las decenas son 3 y las unidades 7. 3 × 16 = 48, y 48 + 7 = 55 = 0x37. El display muestra “37”, es decir, 3,7 segundos.

4.5 Cálculo del promedio

Al terminar la décima ronda, x9 contiene la suma de los diez tiempos en décimas. El promedio, en la misma unidad, es:

```
promedio = parte entera de ((suma + 5) / 10)
```

Se suma 5 antes de dividir para redondear al entero más cercano en vez de truncar. La división por 10 se hace con restas sucesivas, igual que la conversión a décimas.

Ejemplo: si los tiempos fueron 3, 4, 5, 6, 7, 9, 10, 11, 13 y 14 décimas, la suma es 82. Entonces (82 + 5) / 10 = 8,7, cuya parte entera es 8, y el display muestra “08” (0,8 s). Sin el +5 el resultado habría sido 8 también; con una suma de 86 la diferencia se nota: (86 + 5) / 10 = 9,1, que da 9 en vez de 8.

5. Assembler

5.1 Qué hace

El assembler (`assembler/asm.py`) lee un archivo `.s` y genera un `.hex` con una palabra de 32 bits en hexadecimal por línea, que es el formato que lee `$readmemh` en `pochoco_ram.v`. Se usa así:

```
python assembler/asm.py sw/game.s -o sw/game.hex
```

Opciones:

- `-l` muestra un listado con la dirección, el código máquina y la instrucción original de cada línea.
- `-D NOMBRE=valor` cambia el valor de una constante `.equ` sin editar el archivo.

5.2 Sintaxis aceptada

- Una instrucción por línea, con los operandos separados por coma.
- Comentarios con `#`, `//` o `;`.
- Etiquetas con la forma `nombre:`. Pueden ir solas en una línea o antes de una instrucción.
- Constantes con `.equ NOMBRE, valor`.
- Números en decimal, hexadecimal (`0x...`) o binario (`0b...`), con signo negativo opcional.
- Registros `x0` a `x15`, o sus nombres ABI (`zero`, `ra`, `sp`, `t0`, `s0`, `a0`, etc.).
- Se ignoran `.text`, `.section` y `.global`, para que los ejemplos del repositorio también funcionen.

Instrucciones soportadas, agrupadas según su formato de codificación:

Formato	**Instrucciones**
R (registro-registro)	add, sub, slt, sltu, xor, or, and
I (con inmediato)	addi, slti, sltiu, xori, ori, andi, lb, lh, lw, lbu, lhu, jalr
S (store)	sb, sh, sw
B (salto condicional)	beq, bne, blt, bge, bltu, bgeu
U (inmediato alto)	lui, auipc
J (salto)	jal

Pseudoinstrucciones (se traducen a instrucciones reales): `li` (lui + addi), `mv` (addi con 0), `nop` (addi x0, x0, 0), `j` (jal x0), `call` (jal x1), `ret` (jalr x0, 0(x1)), `beqz` y `bnez` (beq y bne contra x0).

5.3 Cómo funciona: dos pasadas

El problema de las etiquetas es que una instrucción puede saltar a una etiqueta que aparece más abajo en el archivo, cuya dirección todavía no se conoce. Por eso el assembler recorre el archivo dos veces.

Primera pasada. Lee línea por línea, quita los comentarios y lleva la cuenta de la dirección actual, empezando en 0 y sumando 4 bytes por cada instrucción. Cuando encuentra una etiqueta, guarda su nombre y su dirección en un diccionario. Cuando encuentra un `.equ`, guarda la constante. En esta pasada no se codifica nada.

Segunda pasada. Codifica cada instrucción. Para los saltos busca la dirección de la etiqueta en el diccionario y calcula el offset como dirección de la etiqueta menos dirección de la instrucción, porque en RISC-V los saltos son relativos al PC.

El caso de li. Para que la primera pasada pueda calcular bien las direcciones, cada instrucción tiene que tener un tamaño conocido. Por eso `li` siempre se traduce a dos instrucciones (lui + addi), aunque el valor sea pequeño.

5.4 Cómo se codifica cada instrucción

Cada formato reparte los 32 bits de manera distinta. El assembler arma la palabra poniendo cada campo en su posición con operaciones de bits:

- **R:** funct7 (31-25), rs2 (24-20), rs1 (19-15), funct3 (14-12), rd (11-7), opcode (6-0).
- **I:** inmediato de 12 bits (31-20), rs1, funct3, rd, opcode.
- **S:** el inmediato se separa en dos partes, bits 11-5 arriba y bits 4-0 donde normalmente va rd.
- **B:** el offset es par, así que el bit 0 no se guarda. Los bits 12, 10-5, 4-1 y 11 van repartidos en posiciones distintas.
- **U:** 20 bits altos del inmediato (31-12), rd y opcode.
- **J:** igual que B, el offset va desordenado: bits 20, 10-1, 11 y 19-12.

Los opcodes y funct3/funct7 son los mismos que reconoce `espino_decoder.v`.

Separación del valor en li. El valor se divide en 20 bits altos para `lui` y 12 bajos para `addi`. El detalle es que `addi` interpreta su inmediato con signo: si el bit 11 de la parte baja es 1, se toma como negativo, y entonces hay que sumar 1 a la parte alta para compensar. Ejemplo con `li x6, 75000000`:

```
75.000.000 = 0x047868C0
parte baja = 0x8C0  -> tiene el bit 11 en 1, como número con signo vale -1856
parte alta = 0x04786 + 1 = 0x04787   (se suma 1 para compensar)

lui  x6, 0x04787      -> x6 = 0x04787000 = 75.001.856
addi x6, x6, -1856    -> x6 = 75.000.000
```

5.5 Ejemplo completo de traducción

La instrucción en la dirección 0x54 de `game.s` es `bltu x3, x4, led_listo`, y la etiqueta `led_listo` está en la dirección 0x74.

1. Es un salto condicional, formato B. Según la tabla del assembler, opcode = 1100011 y para `bltu` funct3 = 110.
2. rs1 = x3 = 00011 y rs2 = x4 = 00100.
3. Offset = 0x74 - 0x54 = 0x20 = 32. En 13 bits: 0 0000 0010 0000. Los campos quedan: imm[12] = 0, imm[10:5] = 000001, imm[4:1] = 0000, imm[11] = 0.
4. Se ordenan según el formato B:

```
imm[12] | imm[10:5] | rs2   | rs1   | funct3 | imm[4:1] | imm[11] | opcode
0       | 000001    | 00100 | 00011 | 110    | 0000     | 0       | 1100011
```

5. Juntando: 0000 0010 0100 0001 1110 0000 0110 0011 = **0x0241E063**.

El listado del assembler muestra lo mismo:

```
0054:  0241e063   bltu x3, x4, led_listo
```

5.6 Validaciones

El assembler se detiene con un mensaje que indica el número de línea cuando encuentra:

- una instrucción que no existe, o un registro mal escrito;
- un registro entre x16 y x31, que no existen en RV32E;
- una cantidad incorrecta de operandos, o un acceso a memoria que no tenga la forma `offset(registro)`;
- un inmediato fuera de rango: 12 bits con signo para los formatos I y S, 13 bits para los saltos condicionales, 21 bits para `jal` y 20 bits sin signo para `lui`;
- una etiqueta no definida o definida dos veces, o una constante no definida;
- un shift (`sll`, `slli`, `srl`, etc.). Estas instrucciones son válidas en RV32E, pero en el Espino se ejecutarían como una suma sin dar ningún error, así que preferimos avisar.

Ejemplo de mensaje: `error: linea 1 ('addi x1, x1, 2048'): inmediato de 12 bits fuera de rango: 2048 (permitido -2048..2047)`.

5.7 Pruebas

El archivo `assembler/test_asm.py` tiene 9 pruebas que se ejecutan con `python assembler/test_asm.py`:

1. Codificación de cada formato (R, I, S, B, U y J), comparada con valores calculados a mano. Incluye el ejemplo `add x9, x12, x13` de la clase y saltos hacia atrás (offset negativo).
2. La traducción de `li` con un valor grande, incluyendo la corrección de signo.
3. El funcionamiento de `.equ` y de `-D`.
4. Que cada uno de los errores de la sección 5.6 sea detectado.
5. Los programas de ejemplo del repositorio (`blink.s`, `7seg.s` y `buttons_leds.s`) vienen con su `.hex` generado con el toolchain oficial de RISC-V. Nuestro assembler genera exactamente los mismos archivos.

Las 9 pruebas pasan. Además, solo para verificar (no se usa en la entrega), comparamos un archivo con todas las instrucciones soportadas contra el assembler GNU de RISC-V y el código máquina fue idéntico.

6. Verificación

6.1 Simulación

El testbench `sim/tb_game.v` simula el SoC completo con iverilog, con el procesador ejecutando el programa real. Simular 3 segundos reales tomaría demasiado tiempo, así que para la simulación ensamblamos el mismo `game.s` con constantes más cortas usando `-D` (por ejemplo 3000 ciclos en vez de 3 s y 100 ciclos por décima). El código es el mismo; solo cambian los números.

Como el iverilog no conoce el bloque de memoria interno de la iCE40 (SB_RAM40_4K), que usa el banco de registros del Espino, agregamos un modelo simple de ese bloque en `sim/sb_ram40_4k_sim.v`. Este archivo solo se usa para simular.

El testbench hace de jugador y juega dos partidas completas. Revisa que:

- la fase con los 4 LEDs dure al menos lo programado;

- después de cada pulsación correcta el display muestre el tiempo esperado;
- al apretar un botón equivocado aparezca “EE” y se repita la misma ronda;
- al final el display muestre el promedio correcto y los LEDs parpadeen;
- las dos partidas produzcan secuencias de LEDs distintas.

Todos los chequeos pasan.

6.2 FPGA

El diseño se sintetiza y rutea sin errores (recursos en la sección 2.4). También verificamos que el contenido de `game.hex` quede cargado en los bloques de memoria del bitstream.

7. Cómo cambiar una constante en la demostración

Por ejemplo, para que la fase inicial dure 5 segundos en vez de 3 (5 × 25.000.000 = 125.000.000 ciclos), hay dos formas:

1. Editar en `game.s` la línea `.equ CICLOS_3S, 75000000` y cambiarla por `125000000`.
2. Sin editar el archivo, usando `-D`:

```
python assembler/asm.py sw/game.s -o sw/game.hex -D CICLOS_3S=125000000
```

Después hay que volver a sintetizar y programar la placa con `make all` (Linux/Mac) o `build.bat` (Windows). De la misma forma se puede cambiar la cantidad de rondas (`RONDAS`) o el tiempo de antirrebote (`CICLOS_REBOTE`).